

Vel Tech Group of Institutions

Name: KISHORE P P

Email: vtu27958@veltech.edu.in

Roll no: vtu27958

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS5L14

VTU_DAA_Week 4_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 24

Section 1 : Coding

1. Problem Statement

You are a delivery manager who needs to plan the delivery route for a fleet of trucks to a set of distribution centers in a city. Each distribution center is at a different location, and the cost of traveling between centers is known.

Your goal is to find the shortest route that allows a truck to visit each distribution center exactly once and return to the starting point, using the Traveling Salesman Problem (TSP). Write a program to calculate the minimum cost of this route and output the optimal path.

Note: Solve using Branch and Bound approach.

Input Format

The first line contains an integer n, the number of distribution centers.

The next n lines contain n space-separated integers each, representing the adjacency matrix.

The ith integer on the jth line represents the cost of traveling from distribution center i to distribution center j.

Output Format

The first line prints:

Minimum Cost is: X

The second line prints:

Optimal Path: followed by the sequence of visited centers ending at the start node

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 3 9 5

3 0 6 4

9 6 0 7

5 4 7 0

Output: Minimum Cost is: 21

Optimal Path: 0 1 2 3 0

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#include <stdbool.h>
```

```
#define MAX 10
```

```

int n;
int adj[MAX][MAX];
int final_path[MAX + 1];
bool visited[MAX];
int final_res = INT_MAX;
int firstMin(int i) {
    int min = INT_MAX;
    for (int k = 0; k < n; k++)
        if (adj[i][k] < min && i != k)
            min = adj[i][k];
    return min;
}
int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;
    for (int j = 0; j < n; j++) {
        if (i == j) continue;
        if (adj[i][j] <= first) {
            second = first;
            first = adj[i][j];
        } else if (adj[i][j] <= second && adj[i][j] != first)
            second = adj[i][j];
    }
    return second;
}

void TSPRec(int curr_bound, int curr_weight, int level, int curr_path[]) {
    if (level == n) {
        if (adj[curr_path[level - 1]][curr_path[0]] != 0) {
            int curr_res = curr_weight + adj[curr_path[level - 1]][curr_path[0]];
            if (curr_res < final_res) {
                for (int i = 0; i < n; i++) final_path[i] = curr_path[i];
                final_path[n] = curr_path[0];
                final_res = curr_res;
            }
        }
    }
    return;
}

for (int i = 0; i < n; i++) {
    if (adj[curr_path[level - 1]][i] != 0 && !visited[i]) {
        int temp_bound = curr_bound;
        curr_weight += adj[curr_path[level - 1]][i];
        if (level == 1)

```

```

        curr_bound -= ((firstMin(curr_path[level - 1]) + firstMin(i)) / 2);
    else
        curr_bound -= ((secondMin(curr_path[level - 1]) + firstMin(i)) / 2);
    if (curr_bound + curr_weight < final_res) {
        curr_path[level] = i;
        visited[i] = true;
        TSPRec(curr_bound, curr_weight, level + 1, curr_path);
    }
    curr_weight -= adj[curr_path[level - 1]][i];
    curr_bound = temp_bound;
    visited[i] = false;
}
}
}

```

```

int main() {
    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);

    int curr_path[MAX + 1];
    int curr_bound = 0;
    for (int i = 0; i < n; i++)
        curr_bound += (firstMin(i) + secondMin(i));

    curr_bound = (curr_bound % 2 == 1) ? curr_bound / 2 + 1 : curr_bound / 2;

    visited[0] = true;
    curr_path[0] = 0;

    TSPRec(curr_bound, 0, 1, curr_path);

    printf("Minimum Cost is: %d\n", final_res);
    printf("Optimal Path: ");
    for (int i = 0; i <= n; i++) {
        printf("%d%s", final_path[i], (i == n) ? "" : " ");
    }
    printf("\n");

    return 0;
}

```

}

Status : Partially correct

Marks : 2/10

2. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n , the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The i th integer on the j th line represents the cost of traveling from gallery i to gallery j .

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30

20 25 30 0

Output: Minimum Cost is: 80

Answer

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>
#define MAX 10
int N;
int cost[MAX][MAX];
bool visited[MAX];
int minCost = INT_MAX;
int firstMin(int i) {
    int min = INT_MAX;
    for (int k = 0; k < N; k++) {
        if (i != k && cost[i][k] < min) {
            min = cost[i][k];
        }
    }
    return min;
}

int secondMin(int i) {
    int first = INT_MAX, second = INT_MAX;
    for (int j = 0; j < N; j++) {
        if (i == j) continue;
        if (cost[i][j] <= first) {
            second = first;
            first = cost[i][j];
        } else if (cost[i][j] < second && cost[i][j] != first) {
            second = cost[i][j];
        }
    }
    return second;
}

void tsp(int currBound, int currWeight, int level, int path[]) {
    if (level == N) {
        if (cost[path[level - 1]][path[0]] != 0) {
            int currRes = currWeight + cost[path[level - 1]][path[0]];
```

```

    if (currRes < minCost)
        minCost = currRes;
    }
    return;
}

for (int i = 0; i < N; i++) {

    if (cost[path[level - 1]][i] != 0 && !visited[i]) {

        int temp = currBound;
        currWeight += cost[path[level - 1]][i];

        if (level == 1)
            currBound -= ((firstMin(path[level - 1]) + firstMin(i)) / 2);
        else
            currBound -= ((secondMin(path[level - 1]) + firstMin(i)) / 2);

        if (currBound + currWeight < minCost) {
            path[level] = i;
            visited[i] = true;
            tsp(currBound, currWeight, level + 1, path);
        }

        currWeight -= cost[path[level - 1]][i];
        currBound = temp;

        for (int j = 0; j < N; j++)
            visited[j] = false;

        for (int j = 0; j < level; j++)
            visited[path[j]] = true;
    }
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            scanf("%d", &cost[i][j]);
}

```

```

int path[MAX];

int currBound = 0;
for (int i = 0; i < N; i++)
    currBound += (firstMin(i) + secondMin(i));

if (currBound % 2 == 1)
    currBound = currBound / 2 + 1;
else
    currBound = currBound / 2;

for (int i = 0; i < N; i++)
    visited[i] = false;

visited[0] = true;
path[0] = 0;

tsp(currBound, 0, 1, path);

printf("Minimum Cost is: %d", minCost);

return 0;
}

```

Status : Partially correct

Marks : 2/10

3. Problem Statement

Sarah, a young entrepreneur starting her e-commerce business, has a limited budget for storing products in her warehouse. Each product has a specific weight and value, and Sarah aims to maximize the total value of the products in her warehouse without exceeding the weight capacity.

To achieve this, Sarah decided to apply the dynamic programming technique, which helps explore potential product combinations and avoid unnecessary calculations.

Input Format

The first line contains a single integer N, representing the number of products.

The next N lines each contain two integers: weight[i] and value[i], where weight[i] is the weight of the product and value[i] is the value (profit) of the product.

The next line contains a single integer W, representing the weight capacity of the warehouse.

Output Format

The output prints a single integer, representing the maximum total profit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5

2 40

3 50

1 100

5 95

3 30

10

Output: 245

Answer

```
#include <stdio.h>
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int N, W;
    if (scanf("%d", &N) != 1) return 0;

    int weights[N];
    int values[N];
    for (int i = 0; i < N; i++) {
        scanf("%d %d", &weights[i], &values[i]);
    }
    if (scanf("%d", &W) != 1) return 0;
    int dp[W + 1];
```

```

for (int i = 0; i <= W; i++) {
    dp[i] = 0;
}
for (int i = 0; i < N; i++) {
    for (int j = W; j >= weights[i]; j--) {
        dp[j] = max(dp[j], dp[j - weights[i]] + values[i]);
    }
}
printf("%d\n", dp[W]);

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

You are a savvy thief preparing to rob a jewelry store. You have a knapsack with a limited capacity and a list of items, each with a certain weight and a corresponding profit.

Your objective is to maximize the total profit you can obtain by selecting items to place in your knapsack while staying within its weight capacity. Write a program to help you calculate the maximum profit you can achieve.

Note: Use the 0/1 knapsack method to solve the program.

Example

Input:

3

60 10

100 20

120 30

50

Output:

220

Explanation:

By selecting the second and third items, you achieve a maximum profit of Rs. 220. The sum of their profits (100 + 120) gives you the highest total profit while not exceeding the knapsack's weight capacity of 50.

Input Format

The first line contains an integer N, representing the number of items available in the store.

Each of the next N lines contains two space-separated integers P and W, where P is the profit that can be obtained by stealing the item and W is the weight of the item.

The last line contains an integer C representing the weight capacity of your knapsack.

Output Format

The output displays an integer representing the maximum total profit that you can achieve by selecting items for your knapsack.

Refer to the sample output for the formatting specifications

Sample Test Case

Input: 3

60 10

100 20

120 30

50

Output: 220

Answer

```
#include <stdio.h>
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    int N, C;
    if (scanf("%d", &N) != 1) return 0;

    int profits[N];
    int weights[N];
    for (int i = 0; i < N; i++) {
        scanf("%d %d", &profits[i], &weights[i]);
    }
    if (scanf("%d", &C) != 1) return 0;
    int dp[C + 1];
    for (int i = 0; i <= C; i++) {
        dp[i] = 0;
    }
    for (int i = 0; i < N; i++) {
        for (int j = C; j >= weights[i]; j--) {
            dp[j] = max(dp[j], dp[j - weights[i]] + profits[i]);
        }
    }
    printf("%d\n", dp[C]);

    return 0;
}
```

Status : Correct

Marks : 10/10